CI/CD Pipelines Documentation
Basic to Expert Level

Hands-on guide for Terraform + Docker + Kubernetes installed on local system

Prepared for: Satish Kumar Mali
Use case: Local DevOps lab, interview preparation, and cloud-ready architecture design
Scope: CI/CD fundamentals, Git, Docker build, Kubernetes deployment, Terraform infrastructure, GitHub Actions, DevSecOps, GitOps, troubleshooting, and production architecture.

Table of Contents

1. What is CI/CD?

CI/CD is a DevOps practice used to automate the software delivery lifecycle from code commit to production deployment.

- CI checks code automatically when developers push changes.
- CD releases verified applications to Dev, Test, UAT, Staging, or Production.
- A pipeline connects build, test, scan, package, deploy, and monitor stages.

Term	Meaning	Example
CI	Continuous Integration	Push code -> build and test automatically
Continuous Delivery	Release is ready but needs approval	UAT or Production approval gate
Continuous Deployment	Release goes automatically	Successful pipeline deploys to production

2. Why CI/CD is Needed

Without CI/CD	With CI/CD
Manual deployment	Automated deployment
Late bug detection	Early testing
No standard release flow	Standard pipeline stages
Hard rollback	Controlled rollback
Slow releases	Fast and reliable releases

3. Real-World Example

For an e-commerce platform, developers update frontend, backend APIs, payment, order, and tracking modules. CI/CD automates release from Git to Kubernetes.

```
Developer -> GitHub -> CI Pipeline -> Docker Image -> Registry -> Kubernetes -> Users
```

- Code push triggers build.
- Pipeline runs tests and security scans.
- Docker image is pushed to registry.
- Kubernetes performs rolling deployment.
- Monitoring validates health and rollback is available.

4. CI/CD Architecture

```
[Developer]
    |
[Git Repository]
    |
[CI Pipeline: Build -> Test -> Scan]
    |
[Docker/Artifact Registry]
    |
[CD Pipeline: Deploy -> Validate -> Monitor]
    |
[Kubernetes / Cloud / Production]
```

Layer	Tools	Purpose

Source Control	GitHub, GitLab, Azure Repos	Code versioning
CI/CD Engine	GitHub Actions, Jenkins, Azure DevOps	Automation
Container	Docker	Portable image
Registry	Docker Hub, ACR, ECR	Store images
Deployment	Kubernetes, Helm, Argo CD	Run application
IaC	Terraform	Provision infrastructure
Security	Trivy, Checkov, Gitleaks	Scan vulnerabilities
Monitoring	Prometheus, Grafana, ELK, Wazuh	Observe health

5. Core Tools and Components

Tool	Role	Learning Focus
Git	Version control	branch, commit, PR
Docker	Image packaging	Dockerfile, build, tag, push
Kubernetes	Orchestration	Deployment, Service, Ingress, HPA
Terraform	Infrastructure as Code	init, plan, apply, state
GitHub Actions	Pipeline as Code	workflow YAML, jobs, secrets
Helm	K8s package manager	charts, values, rollback
Argo CD	GitOps	sync Git desired state to cluster

6. Local Lab Prerequisites

Validate installed tools from PowerShell or terminal.

```
docker --version
docker ps
kubectl version --client
kubectl get nodes
terraform --version
git --version
```

If you do not have a Kubernetes cluster, create one with kind:

```
kind create cluster --name cicd-lab
kubectl get nodes
```

7. Project Folder Structure

```
ci-cd-basic-to-expert/
|-- app/
|   |-- server.js
|   |-- package.json
|   |-- Dockerfile
```

```
|-- k8s/
|   |-- namespace.yaml
|   |-- deployment.yaml
|   |-- service.yaml
|-- terraform/
|   |-- main.tf
|   |-- variables.tf
|   |-- terraform.tfvars
|-- .github/workflows/
|   |-- ci.yml
|   |-- cd-k8s.yml
```

8. Lab 1: Git and Application Setup

```
mkdir ci-cd-basic-to-expert
cd ci-cd-basic-to-expert
mkdir app k8s terraform
cd app
npm init -y
npm install express
notepad server.js
```

server.js

```
const express = require("express");
const app = express();
const port = process.env.PORT || 3000;
const version = process.env.APP_VERSION || "v1";

app.get("/", (req, res) => {
  res.send(`CI/CD Pipeline Working - ${version}`);
});

app.get("/health", (req, res) => {
  res.status(200).json({ status: "healthy", version });
});

app.listen(port, () => console.log(`Application running on port ${port}`));
```

package.json script

```
"scripts": {
  "start": "node server.js",
  "test": "node -e \"console.log('basic test passed')\""
}
```

```
node server.js
# Open: http://localhost:3000
```

9. Lab 2: Dockerize the Application

Dockerfile

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --only=production
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

Build and run

```
cd app
docker build -t cicd-app:v1 .
docker run -d --name cicd-app -p 3000:3000 cicd-app:v1
docker ps
```

```
curl http://localhost:3000/health
docker logs cicd-app
docker stop cicd-app
docker rm cicd-app
```

10. Lab 3: Kubernetes Deployment

namespace.yaml

```
apiVersion: v1
kind: Namespace
metadata:
  name: cicd-dev
```

deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: cicd-app
  namespace: cicd-dev
spec:
  replicas: 2
  selector:
    matchLabels:
      app: cicd-app
  template:
    metadata:
      labels:
        app: cicd-app
    spec:
      containers:
      - name: cicd-app
        image: cicd-app:v1
        imagePullPolicy: IfNotPresent
        ports:
        - containerPort: 3000
        readinessProbe:
          httpGet:
            path: /health
            port: 3000
          initialDelaySeconds: 5
          periodSeconds: 10
        livenessProbe:
```

```
          httpGet:
            path: /health
            port: 3000
          initialDelaySeconds: 15
          periodSeconds: 20
```

service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: cicd-service
  namespace: cicd-dev
spec:
  type: NodePort
  selector:
    app: cicd-app
  ports:
  - port: 80
    targetPort: 3000
```

Apply and test

```
kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl get pods -n cicd-dev
kubectl get svc -n cicd-dev
kubectl port-forward svc/cicd-service 8080:80 -n cicd-dev
```

11. Lab 4: Terraform Workflow

Terraform provisions infrastructure. Locally, use this simple workflow; in cloud, extend it to Resource Group, VNet, AKS, ACR, IAM/RBAC, and monitoring.

```
terraform {
  required_version = ">= 1.5.0"
}

resource "local_file" "pipeline_notes" {
  filename = "${path.module}/pipeline-notes.txt"
  content  = "Terraform workflow: init -> validate -> plan -> apply"
}
```

```
cd terraform
terraform init
terraform fmt
terraform validate
terraform plan
terraform apply -auto-approve
terraform state list
terraform destroy -auto-approve
```

Tool	Responsibility
Terraform	Creates cloud infrastructure
Docker	Creates application image
Kubernetes/Helm	Deploys workloads
CI/CD Pipeline	Automates build, scan, push, deploy

12. Lab 5: GitHub Actions CI Pipeline

```
name: CI - Build and Test
on:
  push:
    branches: [ "main", "dev" ]
  pull_request:
    branches: [ "main" ]

jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with:
        node-version: 20
    - name: Install dependencies
      working-directory: app
      run: npm install
    - name: Run tests
      working-directory: app
      run: npm test
```

```
    - name: Build Docker image
      working-directory: app
      run: docker build -t cicd-app:${{ github.sha }} .
```

13. Lab 6: Docker Registry Pipeline

```
docker login
docker tag cicd-app:v1 <dockerhub-username>/cicd-app:v1
docker push <dockerhub-username>/cicd-app:v1
```

In production use ACR for Azure, ECR for AWS, or GCR/Artifact Registry for GCP.

14. Lab 7: CD to Kubernetes

```
kubectl set image deployment/cicd-app cicd-app=<dockerhub-username>/cicd-app:v1 -n cicd-
  dev
kubectl rollout status deployment/cicd-app -n cicd-dev
kubectl rollout history deployment/cicd-app -n cicd-dev
kubectl rollout undo deployment/cicd-app -n cicd-dev
```

15. Multi-Environment Pipeline

Environment	Branch	Deployment	Approval
Dev	dev	Automatic	No
Test	release/*	Automatic after tests	No
UAT	main	Manual approval	Yes
Production	tag v*	Manual approval and change approval	Yes

```
Dev -> Test -> UAT -> Production
```

16. DevSecOps Scanning

Stage	Tool	Purpose
Code	SonarQube	Quality and bugs
Secrets	Gitleaks/TruffleHog	Find leaked keys/passwords
Image	Trivy	Find image vulnerabilities
Terraform	Checkov/tfsec	Find insecure IaC
Kubernetes	kube-linter	Validate manifests
Runtime	Falco/Wazuh	Detect suspicious activity

```
trivy image cicd-app:v1
checkov -d terraform/
gitleaks detect --source .
```

17. Helm Basics

Helm is a package manager for Kubernetes and makes deployments reusable.

```
helm create cicd-app
helm install cicd-app ./cicd-app -n cicd-dev
helm list -n cicd-dev
```

```
helm upgrade cicd-app ./cicd-app -n cicd-dev
helm rollback cicd-app 1 -n cicd-dev
helm uninstall cicd-app -n cicd-dev
```

18. GitOps with Argo CD Concept

GitOps means Git is the single source of truth. CI builds image; Argo CD syncs Kubernetes manifests from Git.

```
Developer -> Git Push
CI Pipeline -> Build/Test/Scan/Push Image
GitOps Repo -> Update YAML/Helm values
Argo CD -> Sync to Kubernetes
Kubernetes -> Rolling Update
```

Traditional CD	GitOps CD
Pipeline runs kubectl apply	Argo CD pulls desired state from Git
Kubeconfig in pipeline	Cluster access via Argo CD
Push-based	Pull-based
Less audit-friendly	Git history is audit trail

19. Advanced Deployment Strategies

Strategy	Meaning	Use Case
Rolling Update	Gradually replaces old pods	Default Kubernetes
Blue-Green	Two environments and traffic switch	Zero downtime
Canary	Small user percentage first	Risk-controlled release
Feature Flag	Enable feature by config	Business rollout
Rollback	Return to previous version	Failure recovery

20. Production CI/CD Architecture

- Separate dev, test, uat, and prod environments.
- Use private registry such as ACR/ECR/GCR.
- Use Terraform remote backend and state locking.
- Use least privilege and approval gates.
- Scan code, images, IaC, and Kubernetes YAML.
- Validate deployment through monitoring.
- Keep rollback and disaster recovery plan.

```
GitHub/Azure Repos
  -> CI Build/Test/Scan
  -> Docker Image -> ACR/ECR
  -> Helm values update
  -> Argo CD Sync or Pipeline Deploy
  -> AKS/EKS/GKE
  -> Prometheus/Grafana/Wazuh
```

21. Secrets and Access Management

Secret Type	Bad Practice	Good Practice

Docker password	Hardcode in YAML	GitHub Secrets / Key Vault
Kubeconfig	Commit to repo	Secret store and limited service account
Database password	Plain ConfigMap	Kubernetes Secret or Vault
Cloud credentials	Personal admin	Service principal or managed identity

```
kubectl create secret generic db-secret   --from-literal=DB_USER=appuser   --from-
  literal=DB_PASSWORD=StrongPassword123   -n cicd-dev
```

22. Monitoring and Rollback

Monitor	Reason
Pod restart count	Detect crash loops
Rollout status	Confirm release success
HTTP 5xx	Detect app failure
Latency	Detect performance issue
CPU/memory	Detect capacity issue
Security events	Detect suspicious activity

```
kubectl get pods -n cicd-dev
kubectl describe pod <pod-name> -n cicd-dev
kubectl logs <pod-name> -n cicd-dev
kubectl rollout undo deployment/cicd-app -n cicd-dev
```

23. Troubleshooting Guide

Error	Meaning	Check/Fix
ImagePullBackOff	Cannot pull image	Image name, tag, registry auth
CrashLoopBackOff	Container keeps crashing	kubectl logs and startup command
Pending Pod	Pod not scheduled	Node capacity/resources/taints
Port not opening	Service issue	Selector and targetPort
Terraform backend error	State/backend issue	Backend resource and permissions
Docker build failed	Dockerfile/dependency issue	Dockerfile path and npm install
Pipeline secret error	Missing CI secret	Repository settings secrets

24. Beginner to Expert Learning Path

Level	Focus	Practice

Beginner	Git, Docker, basic CI	Build app and image
Intermediate	K8s YAML and rollout	Deploy, scale, rollback
Advanced	Terraform, Helm, DevSecOps	Provision, scan, package
Expert	GitOps, security, governance	Production AKS/EKS design

25. Interview Questions

What is CI/CD?

Automation of build, test, scan, package, deployment, and monitoring.

Continuous Delivery vs Continuous Deployment?

Delivery waits for approval; Deployment releases automatically after checks pass.

Why Docker?

It packages the app and dependencies into a consistent image.

Why Kubernetes?

It provides scaling, self-healing, rolling updates, and service discovery.

Why Terraform?

It provisions infrastructure as code with version control.

How rollback works?

Use kubectl rollout undo, Helm rollback, or previous image tag.

How to secure pipelines?

Secrets manager, least privilege, scanning, approvals, branch protection, audit logs.

26. Architect Statement

I design CI/CD platforms by separating infrastructure, application delivery, and security controls. Terraform provisions cloud infrastructure such as networking, Kubernetes clusters, container registry, IAM/RBAC, and monitoring. Docker packages the application, while Kubernetes or Helm deploys workloads. CI/CD pipelines automate build, test, scan, image push, deployment, rollback, and monitoring validation. For production, I include secrets management, approval gates, DevSecOps scanning, GitOps, multi-environment strategy, observability, disaster recovery, and cost optimization.

27. Daily Practice Plan

Day	Practice
Day 1	Git basics, create app, run locally
Day 2	Dockerfile, image build, container run
Day 3	Kubernetes deployment and service
Day 4	Scale, rollout, rollback, probes
Day 5	Terraform workflow and cloud IaC
Day 6	GitHub Actions CI
Day 7	Registry push and Kubernetes CD

Day 8	Trivy, Checkov, Gitleaks
Day 9	Helm chart and values
Day 10	GitOps and production design

28. Commands Cheat Sheet

```
# Git
git init
git status
git add .
git commit -m "initial commit"
git checkout -b dev
git push origin main

# Docker
docker build -t cicd-app:v1 .
docker run -p 3000:3000 cicd-app:v1
docker images
docker ps
docker logs <container>
docker tag cicd-app:v1 username/cicd-app:v1
docker push username/cicd-app:v1

# Kubernetes
kubectl get nodes
kubectl get pods -A
kubectl apply -f k8s/
kubectl get svc -n cicd-dev
kubectl describe pod <pod-name> -n cicd-dev
kubectl logs <pod-name> -n cicd-dev
kubectl scale deployment cicd-app --replicas=5 -n cicd-dev
kubectl rollout status deployment/cicd-app -n cicd-dev
kubectl rollout undo deployment/cicd-app -n cicd-dev
```

```
# Terraform
terraform init
terraform fmt
terraform validate
terraform plan
terraform apply
terraform state list
terraform destroy

# Helm
helm create cicd-app
helm install cicd-app ./cicd-app -n cicd-dev
helm upgrade cicd-app ./cicd-app -n cicd-dev
helm rollback cicd-app 1 -n cicd-dev
helm uninstall cicd-app -n cicd-dev
```

29. Final Summary

- Basic level means Git, build, test, and Docker image creation.
- Intermediate level means Kubernetes deployment and troubleshooting.
- Advanced level means Terraform, DevSecOps, Helm, secrets, and multi-environment pipelines.
- Expert level means secure, scalable production CI/CD using Terraform, Docker, Kubernetes, GitOps, monitoring, governance, and rollback.

Recommended next project: Build a 3-tier app, containerize frontend and backend, provision AKS with Terraform, push images to ACR, deploy with Helm, and sync production using Argo CD.